7. Bölüm
Fonksiyonlar

7.1 Fonksiyon Nedir?

Yapısal Programlama mantığında çözülecek problem genel olarak fonksiyonların birbirini çağırmasıyla yapıldığı 2.7 başlığında anlatılmıştı. Yani kodlaması yapılacak işi birbirini çağıran fonksiyonlar yazarak ana fonksiyondan bu fonksiyonları çağırarak yaparız. Ayrıca yazılacak programda birbiriyle ilgili fonksiyonlar bir araya toplanarak ayrı bir dosyada tutulur. Bu durum 5.1 başlığı altında anlatılmıştı. Bu bölüme kadar gördüğümüz kodlarda çözümü oluşturan tüm tasarım parçaları ana fonksiyon olan `main()` fonksiyonu içerisine çözülmüştür. Bir bilgisayar programı, genel olarak, tek başına tüm görevleri yerine getiren tek bir main fonksiyonundan oluşmaz. Bu durumda ana problemin büyüklüğüne göre ana fonksiyon bloğumuz uzar, okunabilirliği azalır, müdahale etmek zorlaşır. Bu tip büyük programları kendi içerisinde, her biri özel bir işi çözmek için tasarlanmış parçacıklarından oluşturmak daha mantıklı olacaktır. Yani böl ve yönet (divide and conquer) tekniği uygulanır. Çünkü büyük bir problemin toptan çözülmesi, problemin parçalar halinde çözülmesinden her zaman daha yorucu ve zordur. Yazılım geliştirmede problemi büyük ana parçalara ayırırız. Daha sonra bu büyük parçaları daha küçük parçalara ayırarak çözeriz. Buna yukarıdan aşağıya (top down) yaklaşım adı verilir. C dilinde bunu fonksiyonlar ile yaparız;

- ◇ Fonksiyonlar, belirli bir görevi gerçekleştiren bir kod bloğudur.
- ◇ Parametreli alabilir, girdilerle ya da parametresiz bir şeyler yapar ve ardından cevabı verebilir. Kısaca bilgiyi fonksiyona argümanlar ile aktarırız.
- ◇ Her fonksiyonun bir kimliği vardır. (Değişken Kimliklendirme Kuralları burada da geçerlidir.)
- ◇ Bir fonksiyon program içerisinde istenildiği kadar farklı yerlerden ulaşılarak çalıştırılabilir ya da çağrılabilir. Bir başka deyişle tekrar kullanılabilir.
- ◇ Bir fonksiyon, çağıran koda bir değer döndürebilir.
- ◇ Bir fonksiyon, bir başka fonksiyondan çağrılabilir.

C dilinde iki tip fonksiyon bulunmaktadır; başlık dosyalarında bize sunulan **hazır fonksiyonlar** ve programcının tanımlayacağı **kullanıcı tanımlı fonksiyonlar** (user defined function).

7.2 Hazır Fonksiyonlar

C geliştiricileri, programcıların kullanmaları için, önceden yazılmış olan hazır fonksiyonlar hazırlamışlardır. Hazır fonksiyonlar teknik olarak C dilinin parçası değillerdir. Yalnızca standart hale getirilmişlerdir. Programcı, kullanmak istediği hazır fonksiyon hangi modülde ise o modüle ilişkin **başlık** (header) dosyasını `#include` ön işlemci yönergesi (preprocessor directive) ile kendi projesine dahil eder. Bunu 5.1 başlığı altında incelemiştik. Aslında bu başlık dosyalarında;

- ◇ Hazır fonksiyonların sadece **prototipleri** (prototype) başlık dosyalarında (header file) bulunur. Prototip derleyiciye, parametreleri ve geri dönüş değeri şu olan bir fonksiyon var demenin yoludur.
- ◇ Fonksiyonların kendileri yani derlenmiş halleri her işletim sistemi için ayrı derlenmiş olan kütüphane (library) dosyaları içerisinde bulunur.
- ◇ Derleme işlemi sırasında **bağlayıcı** (linker) program tarafından bu fonksiyonlar icra edilebilir dosyaya (executable file) dahil edilir.

§7.2.1 Math.h Başlık Dosyası

Matematik işlemleri gerçekleştirmek için standart hale getirilmiş bir başlık dosyasıdır. Tablo 7.1'de fonksiyon listesi verilmiştir. Örnek olarak kenarları verilen bir üçgenin alanını hesaplayan bir program aşağıda verilmiştir.

```
#include <stdio.h> //printf ve scanf için dahil ettik.
#include <math.h>  //sqrt için dahil ettik.
int main()
{
        int kenar1, kenar2, kenar3;
        double alan, kenarlarToplamininYarisi;
        printf("Üçgenin Kenarlarını Giriniz:");
        scanf("%d%d%d",&kenar1,&kenar2,&kenar3);
        if ((kenar1+kenar2 <= kenar3) ||
```

```
        (kenar2+kenar3 <= kenar1) ||
        (kenar1+kenar3 <= kenar2) ) {
                printf("Böyle bir üçgen olamaz!\n");
                return 1;
        }
        /*AREA OF A TRIANGLE - HERON'S FORMULA */
        kenarlarToplamininYarisi=(kenar1+kenar2+kenar3)/2.0;
        alan= sqrt( kenarlarToplamininYarisi*
        (kenarlarToplamininYarisi-kenar1)*
        (kenarlarToplamininYarisi-kenar2)*
        (kenarlarToplamininYarisi-kenar3) );
        printf("Ucgenin alani: %.2f",alan );
        return 0;
}
/*Program Çıktısı:
Üçgenin Kenarlarını Giriniz:3 10 12
Ucgenin alani: 12.18
*/
```

Fonksiyon prototipi	Açıklama	Örnek
`double sqrt(double);`	$\sqrt{x}$	`sqrt(900.0)=30.0`
`double exp(double);`	$\exp(x)$	`exp(1.0)=2.718282`
`double log(double);`	$\ln(x)$	`log(2.718282)=1.0`
`double log10(double);`	$log(x)$	`log10(1.0)=0.0`
`double fabs(double);`	$\|x\|$	`fabs(-5.0)=5.0`
`double ceil(double);`	x'i kendisinden büyük en küçük kesirsiz sayıya yuvarlar.	`ceil(-9.8)=-9.0`
`double floor(double);`	x'i kendisinden küçük en büyük kesirsiz sayıya yuvarlar.	`floor(9.8)=9.0`
`double pow(double x,double y);`	x^y	`pow(2.0,3.0)=8.0`
`double fmod(double x,double y);`	Kayan noktalı sayı olarak x'in y'ye bölümünden kalanı verir.	`fmod(13.657,2.333)=1.1992`
`double sin(double);`	Radyan cinsinden x'in sinüsü	`sin(0.0)=0.0`
`double cos(double);`	Radyan cinsinden x'in kosinüsü	`cos(0.0)=1.0`
`double tan(double);`	Radyan cinsinden x'in tanjantı	`tan(0.0)=0.0`

Tablo 7.1: Math.h Başlık Dosyası Fonksiyonları

§7.2.2 Stdlib.h Başlık Dosyası

Bu kütüphane içerisinde; Alfa sayısal metinleri ilkel veri tiplerine veya bunun tersini yapan tür dönüşüm fonksiyonları (`atoi()`, `itoa()`, `atof()`, `strtod()`, ... ileride anlatılacaktır.), Çalıştırma anında (run time) bellek tahsisi ve tahsisli belleğin serbest bırakılmasını sağlayan hafıza yerleşim fonksiyonları (`malloc()`, `free()` ... ileride anlatılacaktır.) Rastgele sayı üretme fonksiyonları (`rand()`, `srand()`) bulunur. Şimdilik rastgele sayı üretme üzerinde durulacaktır. Aşağıda bir zar için rastgele sayı üreten bir program verilmiştir.

Fonksiyon	Açıklama
`int rand();`	`0` ile `RAND_MAX` yani `32767` arasında rastgele bir sayı üretir.
`int srand(unsigned int);`	`rand()` fonksiyonu belli bir başlangıç değerinden itibaren bir dizi matematiksel işlem sonucu rastgele bir değer üretir. `srand()` fonksiyonu, `rand()` fonksiyonuna başlangıç değerini farklı vermek için kullanılır.

Tablo 7.2: Stdlib.h Başlık Dosyasının Bazı Fonksiyonları

```
#include <stdio.h> // printf için
#include <stdlib.h> //rand için
int main() {
```

```
        int rastgeleZar;
        rastgeleZar=1 + rand() %6;
        /* rand() fonksiyonunun üreteceği sayıyı kalan işleci ile 6 sayısına böleriz ki 0 ile 5
         ↪  arasında bir sayı elde edelim. Buna da 1 ekler isek zarın rakamları ortaya çıkar.*/
        printf("Atılan Zar:%d",rastgeleZar);
        return 0;
}
```

Fakat programın her çalışmasında `rand()` aynı başlangıç değerini kullandığından aynı rastgele değerleri üretir. Bu nedenle aynı zar rakamı gelmiş olur. Bunu değiştirmek için `srand()` fonksiyonu kullanılır.

```
#include <stdio.h>  //printf için
#include <stdlib.h> //rand ve srand için
int main() {
        int rastgeleZar;
        int randBaslangicDegeri;
        printf("rand() için başlangıç değeri olabilecek bir sayı giriniz:");
        scanf("%d",&randBaslangicDegeri);
        srand(randBaslangicDegeri);
        rastgeleZar=1 + rand() %6;
        printf("Atılan Zar:%d",rastgeleZar);
        return 0;
}
```

Rastgele sayı her seferinde kullanıcıdan alınan sayıya göre hesaplandığından her seferinde farklı bir zar rakamı üretilmiş olacaktır. **Ancak başlangıç değerinin her seferinde kullanıcıdan alınması bir başka problemdir**. Bunun üstesinden gelmek için bilgisayarın saatini sayı olarak veren `time.h` başlık dosyasındaki `time()` fonksiyonu `NULL` parametresiyle kullanılabilir.

```
#include <stdio.h>  // printf için
#include <stdlib.h> // rand ve srand için
#include <time.h>   // time fonksiyonu için
int main() {
        int rastgeleZar;
        int randBaslangicDegeri=time(NULL);
        srand(randBaslangicDegeri);
        // üst iki satır yerine "srand(time(NULL));" yazılabilir!
        rastgeleZar=1 + rand() %6;
        printf("Atılan Zar:%d",rastgeleZar);
        return 0;
}
```

7.3 Kullanıcı Tanımlı Fonksiyonlar

Programcılar kendi fonksiyonlarını oluşturarak kodlarını daha kullanışlı hale getirirler. Bu tür fonksiyonlara **kullanıcı tanımlı fonksiyonlar** (user defined function) denilir.

Şimdiye kadar yazdığımız kodlarda ana fonksiyonumuz olan `main()` hep `sqrt()`, `rand()` gibi fonksiyonları çağırmıştır. Çağıran program, kullanılacak fonksiyonu bilmelidir. Kullandığımız başlık dosyalarında aşağıda açıklayacağımız **fonksiyon bildirimleri** (function declaration) bulunur.

Bir fonksiyonu çalıştıran koda, **çağıran program** adı verilir. 7.2 Başlığındaki örneklerde `main()` ana fonksiyonumuz çağıran programdır.

Kullanıcı tanımlı fonksiyonu oluşturmak ve kullanmak için bu dört unsuru bilmemiz gerekir;

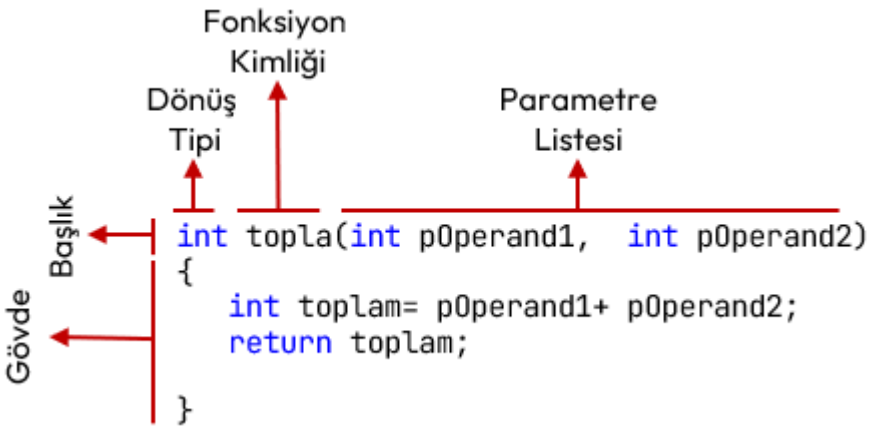
1. **Fonksiyon Tanımı** (function definition): **Bir fonksiyonun girdileri, yaptığı işlem ve döndüreceği değeri içerecek şekilde başlık ve gövdesinin kodlanmasından oluşur.** Fonksiyon tanımı, bildirimi yapılan fonksiyonu, derleme yapılabilmesi için eksiksiz bir şekilde tamamlar. Yani, başlık ve süslü parantez içerisinde yer alan kodlardan oluşur.
2. **Fonksiyon Bildirimi** (function declaration): **Derleyiciye, programın ilerleyen kısımlarında veya belki başka bir dosyada verilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler.** Derleyici bu sayede, fonksiyonun gövdesini görmese bile onun nasıl çağrılacağını; dönüş tipi ve argümanlarını bilir. Bildirimin amacı, derleyiciye "Böyle bir fonksiyon var, tipi budur," bilgisini vermektir
3. **Fonksiyon Prototipi** (function prototype): **Aslında fonksiyon bildirimi ile aynı şeydir.** An-

cak "prototip" terimi genellikle fonksiyonun kodun en tepesinde, `main()` fonksiyonundan önce yapılan bildirimini ifade etmek için kullanılır. Bir fonksiyonu tanımlandığı satırdan daha yukarıda bir yerde çağırmak istiyorsanız, en tepeye bir prototip/bildirim eklemek zorundasınızdır. Aksi takdirde derleyici hatası (identifier not found) verir.

4. **Fonksiyon Çağırma**(function call/invoke function): Çağıran program içinde, bildirimi daha önce yapılmış bir fonksiyon, kimliğini ve ardından parantez içindeki argüman listesini yazarak çağırabiliriz.

§7.3.1 Fonksiyon Tanımı Nasıl Yapılır?

Fonksiyon tanımı (function definition), **fonksiyonun girdileri, kimliği, yaptığı işlem ve döndüreceği değeri içerecek şekilde başlık ve gövdesinin kodlanmasından oluşur.**



Şekil 7.1: Fonksiyon Tanımı Örneği

Şekilde iki `int` parametresi olan ve `int` veri tipinde geri değer döndüren bir `topla` fonksiyonunun tanımlaması verilmiştir. Fonksiyon tanımlama örneğini aşağıdaki şekilde açıklayabiliriz;

- Fonksiyon bir değer döndürebilir. **Dönüş tipi**, işlevin döndürdüğü değerin veri tipidir. Değer döndürmeyecek ise dönüş tipi `void` olarak yazılır.
- Fonksiyonun kimliği benzersizdir. Aynı zamanda **fonksiyon adı** (function name) olarak adlandırılır. Fonksiyon kimliklendirmede, 4.4 başlığındaki değişken kimliklendirme kuralları geçerlidir.
- Fonksiyon kimliğinden sonra **parametreleri barındıracak açıp kapatılan parantez olmalıdır**. Bu durum değişken tanımından fonksiyonları ayırmak içindir.
- Bağımsız değişkenler parametre olarak adlandırılır. **Parametre listesi**, bir işlevin parametrelerinin veri tipini, sırasını ve sayısını belirtir. Parametreler isteğe bağlıdır; yani bir fonksiyon hiçbir parametre içermeyebilir. Fonksiyon tanımında **her bir parametreye kimlik verilmelidir**. Çünkü fonksiyon gövdesinde bu değişkenler işlenir.
- **Fonksiyon Gövdesi**, fonksiyonun ne yaptığını tanımlayan talimatları içerir. Yapısal programlama yapıldığından ilk önce değişkenler tanımlanmalı daha sonra bu değişkenleri işleyecek talimatlar yazılmalıdır. Fonksiyonda istenilen sonuca ulaşmak için kullanacağı adımlara karar verilir yani algoritması belirlenir.
- **Dönüş tipine** uygun olarak bir değer, `return` talimatı ile geri döndürülmelidir. `return` talimatı, bir fonksiyonun yürütülmesini sonlandırır ve kontrolü çağrıldığı fonksiyona geri verir. Yani programın icrasına devam etmek için çağrıldığı yere dönülür. **Bir fonksiyonun tanımında yer alan dönüş tipi ile `return` talimatına ilişkin ifade sonucu aynı veri tipinde olmalıdır.** Hiçbir değer geri döndürülmeyecek ise geri dönüş tipi `void` olarak belirlenir. Bu durumda `return;` talimatı sonrasında bir değer yer almaz. `return` talimatı çoğunlukla fonksiyonundaki son talimat olarak görünür.
- Fonksiyonun çağrıldığı yerde, fonksiyonun döndürdüğü değerin tipi ile eşleşen bir değişkene dönüş değeri atanabilir. Zorunlu değildir.
- **C dilinde bir fonksiyon gövdesinde bir başka fonksiyon tanımlanamaz!**

Aşağıda `topla` adlı bir kullanıcı tanımlı fonksiyona ilişkin program verilmiştir.

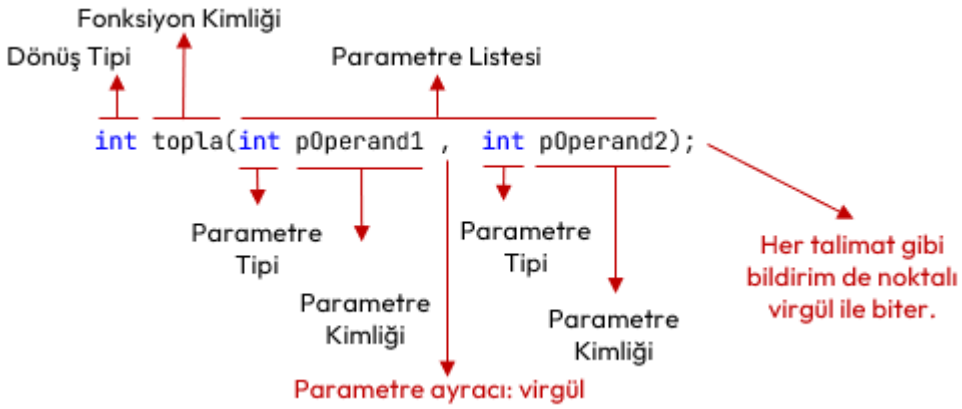
```
#include <stdio.h>
// Kullanıcı tanımlı "topla" fonksiyonu burada tanımlanıyor (definition):
int topla(int pOperand1,int pOperand2) //Fonksiyon başlığı
{ //Fonksiyon bloğu başlangıcı
```

```
        //Fonksiyon Gövdesi:
        int toplam=pOperand1+ pOperand2;
        return pOperand1+ pOperand2; // Toplanan tamsayılardan ortaya çıkan tamsayı geri
         ↪ döndürülüyor!
} //Fonksiyon bloğu bitişi
//Bu noktadan sonra topla kimlikli fonksiyon çağrılabilir.
int main () {
        // ÇAĞRI ORTAMI:
        /* Ana fonksiyon içinde "topla" fonksiyonu çeşitli şekillerde ÇAĞRILACAK! (CALL). */
        int sonuc;
        topla(1,2); /* topla fonksiyonu çağrılıyor ama geri döndürdüğü değer kullanılmıyor. */
        printf("toplam: %d\n", topla(2,3)); /* topla fonksiyonu çağrılıyor. Geri döndürdüğü değer
         ↪ printf fonksiyonuna argüman olarak giriyor. */
        sonuc= topla(3,4); /* topla fonksiyonu çağrılıyor. Geri döndürdüğü değer sonuc
         ↪ değişkenine atanıyor. */
        printf("toplam: %d\n", sonuc);
        sonuc= topla(1,2)+topla(3,4); /* topla fonksiyonu iki kez çağrılıyor. Her bir çağrıdaki
         ↪ geri dönüş değeri geçici bir yerde saklanıp toplamı sonuc değişkenine atanıyor. */
        printf("toplam: %d\n", sonuc);
        sonuc= topla(topla(1,2),3)+topla(4,5); // Bir fonksiyon birden fazla çağrılabilir!
        printf("toplam: %d\n",sonuc);
        return 0;
}
/*Program Çıktısı:
toplam: 5
toplam: 7
toplam: 10
toplam: 15
*/
```

Örnek programda görüldüğü üzere 2.7 başlığında bahsedilen yapısal programın iskeleti oluşmuştur. Ana program içinde problemin çözümü, çeşitli fonksiyonların birbirini çağırması ile yapılmıştır. Her bir fonksiyon içinde işlenecek veriye ilişkin değişkenler (en basit veri yapısı) tanımlanmış ve bu veriler ile veri yapılarını işleyen kontrol işlemleri birbirinden ayrılmıştır. Okunaklılık çok yüksek bir kod elde edilmiştir.

§7.3.2 Fonksiyon Bildirimi Nasıl Yapılır?

Fonksiyon bildirimi (function declaration), derleyiciye, programın ilerleyen kısımlarında veya belki başka bir dosyada verilen kimlikte ve verilen parametrelere sahip bir fonksiyonun var olduğunu söyler.



Şekil 7.2: Fonksiyon Bildirimi Örneği

Şekil 7.2'de iki `int` parametresi olan ve `int` veri tipinde geri değer döndüren `topla` kimlikli bir fonksiyon bildirimi örneği verilmiştir. Bildirim örneğini aşağıdaki şekilde açıklayabiliriz;

- **Dönüş tipi**, fonksiyonun döndüreceği değerin veri tipidir.
- **Fonksiyonun kimliği** benzersizdir.
- Fonksiyon kimliğinden sonra parametreleri barındıracak **açıp kapatılan parantez** olmalıdır. Bu durum fonksiyon bildirimini, değişken tanımından ayırmak içindir.
- **Parametre tipi**, fonksiyon girdisi olan parametrelerin yani bağımsız değişkenlerin veri tipidir. **Bildirimdeki parametrelere kimlik vermek zorunlu değildir!**

- ◇ Parametreler birden çok olabilir. Bu durumda aralarına **virgül işleci** (comma operator) konulur.
- ◇ Her talimat gibi fonksiyon bildirimi de **noktalı virgül** ile biter!

7.3.1 Başlığındaki `topla` örneğimizi bildirim yapılarak aşağıdaki şekilde yazılabilir;

```
#include <stdio.h>
int topla(int,int); /* kullanıcı tanımlı topla fonksiyonu bildirimi (declaration): Bu bildirimde
 ↪ int geri döndüren ve int titpinde iki farklı parametresi olan "topla" kimlikli fonksiyon
 ↪ derleyiciye bildirildi.*/
int main () {
        // ÇAĞRI ORTAMI:
        /* Ana fonksiyon içinde "topla" fonksiyonu çeşitli şekillerde ÇAĞRILACAK! (CALL). */
        int sonuc;
        topla(1,2); /* topla fonksiyonu çağrılıyor ama geri döndürdüğü değer kullanılmıyor. */
        printf("toplam: %d\n", topla(2,3)); /* topla fonksiyonu çağrılıyor. Geri döndürdüğü değer
         ↪ printf fonksiyonuna argüman olarak giriyor. */
        sonuc= topla(3,4);  /* topla fonksiyonu çağrılıyor. Geri döndürdüğü değer sonuc
         ↪ değişkenine atanıyor. */
        printf("toplam: %d\n", sonuc);
        sonuc= topla(1,2)+topla(3,4); /* topla fonksiyonu iki kez çağrılıyor. Her bir çağrıdaki
         ↪ geri dönüş değeri geçici bir yerde saklanıp toplamı sonuc değişkenine atanıyor. */
        printf("toplam: %d\n", sonuc);
        sonuc= topla(topla(1,2),3)+topla(4,5); // Bir fonksiyon birden fazla çağrılabilir!
        printf("toplam: %d\n",sonuc);
        return 0;
}
// Yukarıda Bildirimi yapılmış kullanıcı tanımlı "topla" fonksiyonu burada tanımlanıyor
 ↪ (definition):
int topla(int pOperand1,int pOperand2) //Fonksiyon başlığı
{ //Fonksiyon bloğu başlangıcı
        int toplam=pOperand1+ pOperand2;
        return pOperand1+ pOperand2; // Toplanan tamsayılardan ortaya çıkan tamsayı geri
         ↪ döndürülüyor!
} //Fonksiyon bloğu bitişi

/*Program Çıktısı:
toplam: 5
toplam: 7
toplam: 10
toplam: 15
*/
```

> Daha önce fonksiyonun bildirimi (function declaration) yapılmış ise fonksiyon tanımı (function definition) aynı kimlikle tanımlanmalıdır. Ayrıca 7.3.1 başlığında verilen örnekteki gibi **hiç bildirim yapılmadan da fonksiyonlar tanımlanabilir**. Ancak fonksiyon tanımlandığı yerden sonra çağrılabilir.

Aşağıdaki fonksiyon bildirimi içeren bir başka programı inceleyelim. Bu program derleyici tarafından derlenir ancak **bağlama** (linker) aşamasında hata verir. Çünkü **fonksiyon tanımlamaları** (function definition) yapılmamıştır.

```
int ikiSayiyiTopla(int,int); /* "ikiSayiyiTopla" kimlikli bir fonksiyon olduğu ve iki tamsayı
 ↪ parametre aldığı ve ayrıca bir tamsayı geri döndürdüğü derleyiciye bildiriliyor. */

void sayiyiKonsolaYaz(int);  /* "sayiyiKonsolaYaz" kimlikli bir fonksiyon olduğu ve bu fonksiyonun
 ↪ bir tamsayı parametre aldığı ve geri değer döndürmediği derleyiciye bildiriliyor. */

int konsoldanTamsayiOku();   /* "konsoldanTamsayiOku" kimlikli bir fonksiyon olduğu ve bu
 ↪ fonksiyonun bir parametre almadığı ve bir tamsayı geri döndürdüğü derleyiciye bildiriliyor. */

int main() {
        int sayi;
        sayi=konsoldanTamsayiOku();
        sayiyiKonsolaYaz(13);
```

```
        sayiyiKonsolaYaz(sayi);
        int toplam= ikiSayiyiTopla (sayi,20);
        //...
        return 0;
}
```

Fonksiyona ilişkin bir bildirim yapıldığında bir yerlerde böyle bir fonksiyon olduğu derleyiciye iletilmiş olur. Bildirim sonrasında artık onu çağırabiliriz. Ancak fonksiyon tanımı yapılmamış ise bağlayıcı (linker) program derlemeyi tamamlayamaz.

Program ana fonksiyondan başladığından her programcı ana fonksiyona odaklanır. Bu nedenle **ilk önce fonksiyon bildirimleri yapılıp fonksiyon tanımlarının ana fonksiyon sonrasına bırakılması etik bir davranıştır**.

Aşağıda pozitif sayı girilmesini garanti ettikten sonra sayının 10 sayısına bölünebilirliğini bulan program verilmiştir.

```
#include <stdio.h>
int pozitifSayiOku(); /* parametre almayan bir fonksiyon bildirimi (declaration). Bu fonksiyon
↪ geri int değer döndürüyor. */
int onaBolunurMu(int); /* int veri tipinde parametre alan bir başka fonksiyon bildirimi
↪ (declaration): bu fonksiyonda int tipinde veri geri döndürüyor. */
int main () {
        int birPozitifSayi=pozitifSayiOku();
        if (onaBolunurMu(birPozitifSayi))
                printf("Girilen Pozitif Sayı 10 ile bölünebilir.");
        else
                printf("Girilen Pozitif Sayı 10 ile tam bölünmez.");
        return 0;
}
/* Bildirimi yapılan "pozitifSayiOku" fonksiyonunun tanımı (definition): */
int pozitifSayiOku() {
        int okunan;
        do {
                printf("Lütfen pozitif sayı giriniz: ");
                scanf("%d",&okunan);
                if (okunan<=0)
                printf("HATA:Pozitif Sayı GİRMEDİNİZ!\n");
        } while (okunan <= 0);
        return okunan;
}
/* Bildirimi yapılan "onaBolunurMu" fonksiyonunun tanımı (definition): */
int onaBolunurMu(int p){
        return (p%10)==0;
}
/*Program Çıktısı:
Lütfen pozitif sayı giriniz: 30
Girilen Pozitif Sayı 10 ile bölünebilir.
*/
```

Bunların dışında hiçbir değer geri döndürmeyen fonksiyonlar da tanımlayabiliriz. Bunlara diğer dillerde prosedür (procedure) adı da verilir. Bu durumda değeri olmayan veri tipi olan `void` anahtar kelimesi kullanılır. Bu tür fonksiyonların da geri dönüş talimatı yalnızca `return;` şekilde yazılır.

```
#include <stdio.h>
void printMenu(); /* değer geri döndürmeyen bir fonksiyon bildirimi (declaration) aynı zamanda
↪ parametre de almıyor. */
int main() {
        int secim;
        do {
                printMenu();
                scanf("%d",&secim);
                switch(secim) {
                case 1:
```

```
                        printf("Verileri Sakladım.\n");
                        break;
                case 2:
                        printf("Verileri Okudum.\n");
                        break;
                case 0:
                        printf("Programdan Çıkılıyor...\n");
                        break;
                default:
                        printf("Tekrar Seçim Yapınız!\n");
                }
        } while (secim!=0);
        return 0;
}
/* Bildirimi yapılan "printMenu" fonksiyonunun tanımı (definition): */
void printMenu() {
        printf("\n");
        printf("1-Verileri Yaz.\n");
        printf("2-Verileri Oku.\n");
        printf("0-ÇIKIŞ.\n");
        return;
}
```

Bir başlık(header) dosyasında bildirimi yapılan bir fonksiyon, C derleyicisi tarafından dahil edileceği kaynak dosyaların tamamında yalnızca bir kez tanımlanabilir. Buna **tek tanım kuralı** (one definition rule) adı verilir. Kısaca birden çok dosya için derleme söz konusu olduğunda birden çok bildirim olabilir ama 1 adet fonksiyon tanımı olmalıdır. İleride detayı verilecektir.

7.4 Çağrı Kuralı

Çağrı kuralı (calling convention), bir fonksiyon çağrısıyla karşılaşıldığında argümanların fonksiyona hangi sıraya göre iletileceğini belirtir. İki olasılık vardır; Birincisi argümanlar soldan başlanarak sağa doğru fonksiyona geçirilir. İkincisi ise C dilinde kullanılır ve **argümanlar sağdan başlanarak sola doğru fonksiyona geçirilir.**

```
fonksiyon(a,b,c,d,e);
```

Yukarıdaki fonksiyon çağrısında argümanların hangi sırada fonksiyona geçirildiğinin bir önemi yoktur. Ancak aşağıdaki verilen kod örneğindeki durumu inceleyelim;

```
int a = 1; printf("%d %d %d", a, ++a, a++);
```

Bu kodun çıktısı `1 2 3` olarak beklenir ancak çağrı kuralından ötürü çıktı `3 3 1` olur. **Bunun nedeni C dilinin çağrı kuralının sağdan sola olmasıdır.** Bunu şöyle açıklayabiliriz;

1. Fonksiyona sağdan ilk argüman olarak a değişkeninin değeri 1 geçirilir.
2. Ardından a++ ifadesi ile a değişkeni 2'ye yükseltilir.
3. Sonra ++a ifadesi ile a değişkeni 3'e yükseltilir.
4. Fonksiyona ikinci argüman olarak 3 geçirilir.
5. Fonksiyona son olarak a değişkeninin son değeri olan 3 geçirilir.

7.5 Depolama Sınıfları

Depolama sınıfları (storage classes) bir değişkenin ömrünü (lifetime), görünürlüğünü (visibility), bellek konumunu (memory location) ve başlangıç değerini (initial value) belirlemek için kullanılır. Bu özellikler temel olarak bir programın çalışma süresi boyunca belirli bir değişkenin varlığını izlememize yardımcı olan **kapsam** (scope), görünürlüğü ve yaşam süresini içerir.

Yapısal programlama ve fonksiyon kavramının bilgisayarlarda kullanılmasıyla birlikte işlemcilerde de değişiklikler olmuştur. Şekil 1.3'de anlatılan işlemciye yeni kaydediciler de eklenmiştir;

- Verilerle icra edilen kod farklı bellek bölgesinde bulunur. Bu nedenle verileri işaret eden **veri adresleri kaydedicisi** (data memory register).
- **Yığın bellek kaydedicisi** (stack register), fonksiyon çağrıları sırasında mevcut işlemci durumu ve yerel değişkenler gibi bazı işlemleri depolamak için kullanılan belleği gösteren (stack pointer) adresi tutan kaydedicidir.

- **Bayrak kaydedicisi** (flag register), işlemcinin durumunu veya sıfır (zero) bayrağı, taşıma (carry) bayrağı, işaret (sign) bayrağı, taşma (overflow) bayrağı, eşlik (parity) bayrağı, yardımcı taşıma (auxiliary carry) bayrağı ve kesme etkinleştirme (interrupt enable) bayrağı gibi çeşitli işlemlerin sonucunu tutmak için kullanılan özel bir kayıt türüdür.
- **Genel amaçla kullanılan kaydediciler** (general purpose registers).

Eklenen bu kaydediciler ile derlenen her bir program, dört bellek bölgesinden (segment) oluşur;

1. **Kod bellek** ya da kaynak koda ithafla **metin bellek** (code segment/text segment) icra edilecek emirleri içeren bellek.
2. Programın işleyeceği verileri tutan **veri belleği** (data segment).
3. Fonksiyonları için kullanılan yığın bellek (stack segment). Bu bellek bölgesi, son giren veri ilk çıkar LIFO (last in first out) mantığıyla çalışır. Çünkü;
   - **Çağrıdan** (call) önce mevcut işlemci durumu ve yerel değişkenler gibi bazı işlemleri depolamak ve
   - Fonksiyondan dönülünce (return) işlemciyi ve çağrı ortamını eski hale getirmek için kullanılır.
4. Programın icrası sırasında dinamik olarak eklenip çıkarılan veriler için hurdalık gibi kullanılan öbek bellek (heap segment). İleride dinamik bellek yönetiminde anlatılacaktır.

Depolama sınıfları, tanımlanan değişkenlerin hangi bellek bölgesinde yer alacağını belirler. Dört tür depolama sınıfı vardır. Aşağıda başlıklar halinde verilmiştir.

§7.5.1 auto Depolama Sınıfı

Otomatik depolama sınıfı (auto storage class), bir fonksiyon veya blok içinde kimliklendirilmiş tüm yerel değişkenler için varsayılan (default) depolama sınıfıdır. Bu nedenle, C dilinde program yazarken `auto` anahtar sözcüğü nadiren kullanılır. Otomatik değişkenlere;

- **Yalnızca tanımlı olduğu blok içinden erişilebilir.** Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır.
- Yığın bellek bölgesinde (stack segment) tutulur.

```c
#include <stdio.h>
int main() {
        auto int a = 32;
        int b=20;
        /* "int a=32;" talimatı ile eşdeğerdir.  a ve b değişkenleri bu fonksiyon bloğu ve iç
         ↪  bloklarda geçerlidir. Yani faaliyet alanları (scope) bu fonksiyon ile sınırlıdır. */
        if (a==32){
                int b=10; /* Ana fonksiyon bloğunda tanımlı b değişkenine artık ulaşılamaz. */
                int c=50;
                /* "auto int b;" talimatı ile eşdeğerdir. if bloğunda ve tanımlanabilecek iç
                 ↪  bloklarda geçerlidir. */
                a=16;// Bu blokta a da geçerlidir.
                printf("a:%d, b:%d c:%d\n",a,b,c); // Çıktı: a:16, b:10 c:50
        }
        /* if bloğu dışında sadece ana fonksiyon bloğunda tanımlı değişkenlere ulaşılabilir. */
        printf("a:%d, b:%d\n", a,b); // Çıktı: a:16, b:20
        return 0;
}
```

§7.5.2 static Depolama Sınıfı

Statik depolama sınıfı (static storage class) yaygın olarak kullanılan statik değişkenleri (static variable) saklamak için kullanılır. Statik değişkenler;

- **Yalnızca tanımlı olduğu blok içinden erişilebilir.** Yani değişkenin kapsamı, içinde bulunduğu blok ile sınırlıdır.
- Kodun yazılı olduğu dosyanın başında tanımlı static değişkenlere dosya içerisinde her yerden erişilebilir.
- **Statik değişkenler, kapsam (scope) dışına çıktıktan sonra bile değerlerini koruma özelliğine sahiptir.**
- Evrensel (global) tanımlanması halinde, programın herhangi bir yerinden erişilebilir.
- Veri bellek bölgesinde (data segment) yer tahsis edilir. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler (initial value) hep sıfırdır.

Statik olarak tanımlanan değişken (static varşable), program başladığında veri bellek bölümünde oluşturulup program bitene kadar aynı bellek bölgesini işgal eden değişkendir.

```
#include <stdio.h>
static int durum=10; /* Bu static değişkene bu kodun bulunduğu dosyada her yerden erişilebilir. */
void func() {
        int sayac = 0;
        durum++;
        for (sayac = 1; sayac < 5; sayac++)
        {
                static int statikOlan = 5;  /* Buraya her erişildiğinde son değeri ile işlem
                ↪  görür. */
                int statikOlmayan = 10;
                statikOlan++;
                statikOlmayan++;
                durum++;
                printf("sayaç: %d, staticOlan: %d, durum: %d\n", sayac,statikOlan,durum);
                printf("sayaç: %d, statikOlmayan: %d, durum %d\n", sayac, statikOlmayan,durum);
        }
        //Burada "statikOlan" değişkene erişilemez.
        durum++;
        printf("durum: %d\n", durum);
        return;
}
int main() {
        func();
        return 0;
}
/*Program Çıktısı:
sayaç: 1, staticOlan: 6, durum: 12
sayaç: 1, statikOlmayan: 11, durum 12
sayaç: 2, staticOlan: 7, durum: 13
sayaç: 2, statikOlmayan: 11, durum 13
sayaç: 3, staticOlan: 8, durum: 14
sayaç: 3, statikOlmayan: 11, durum 14
sayaç: 4, staticOlan: 9, durum: 15
sayaç: 4, statikOlmayan: 11, durum 15
durum: 16
*/
```

§7.5.3 extern Depolama Sınıfı

Harici depolama sınıfı (extern storage class) bize basitçe değişkenin kullanıldığı blokta değil başka bir yerde tanımlandığını söyler. Harici tanımlanan değişkenler;

- Değerler tanımlandığı yerde değil farklı bir yerde atanır ve üzerinde değişiklik yapılabilir.
- Bir değişkene `extern` anahtar sözcüğü sıfat olarak verildiğinde evrensel (global) değişken olur. Böyle tanımlanan değişkene programın her yerinden erişilebilir.
- Bu değişkenler de veri bellekte (data segment) yer alır. Derleme sırasında bu bellek hep sıfırla doldurulduğundan ilk değerler hep sıfırdır.

Bir harici değişkene `extern` sıfatı verildiğinde **harici değişken** (extern variable) adını alır ve bir başka dosyada tanımlanmış olduğu kabul edilir. Buna örnek olarak aşağıdaki iki farklı kaynak kod dosyası gereklidir. Birinci kaynak kod dosyası (`xtrn.c`) aşağıda verilmiştir.

```
//XTRN.C:
int externDegisken; //Diğer dosyalarda harici kullanılacak değişken tanımlandı.
void funcExtern() //Diğer dosyalarda harici kullanılacak fonksiyon tanımlandı.
{
        externDegisken++;
}
```

Bu dosyadaki değişken ve fonksiyonları kullanan bir başka kaynak kod `main.c` aşağıda verilmiştir;

```
//MAIN.C:
#include <stdio.h>
extern void funcExtern(); /* bir başka dosyada tanımlandı. Burada sadece bildirimi yapıldı.
↪  Böylece evrensel bir "funcExtern" fonksiyonumuz oldu */
int main()
```

```
{
        extern int externDegisken; /* bir başka dosyada tanımlandı. Burada sadece bildirimi
        ↪  yapıldı. Böylece evrensel bir "externDegisken" değişkenine erişim sağlamış olduk */
        funcExtern();
        printf("extern: %d\n", externDegisken);
        externDegisken = 2;
        printf("extern: %d\n", externDegisken);
        return 0;
}
```

Bu iki dosyayı birlikte derlemek için `gcc xtrc.c main.c -o main.exe` komutu ile derleyiciye iki farklı dosya parametre olarak verilir.

§7.5.4 register Depolama Sınıfı

Kaydedici depolama sınıfı (register storage class), otomatik değişkenlerle aynı işlevselliğe sahiptir. Tek fark, derleyicinin, eğer boş bir işlemci kaydedicisi (register) mevcutsa değişken olarak o kaydedicinin kullanılmasıdır. Genel amaçlı kaydediciler bu amaçla kullanılır. Bu da bu değişkenlerin, bellekte saklanan değişkenlerden çok daha hızlı olmasını sağlar. Bu değişkenler textttregister anahtar sözcüğüyle tanımlanırlar;

- ◇ Eğer boş bir CPU kaydedicisi mevcut değilse, değişken yalnızca bellekte saklanır.
- ◇ Sadece bloklar içinde tanımlanan yerel (local) değişkenler `register` sıfatı ile tanımlanır. Evrensel (global) değişkenler bu anahtar sözcük ile kullanılamaz.

Aşağıda kaydedici bu depolama sınıfına ilişkin örnek verilmiştir. Buradaki sayma ve toplama işlemleri uygun olan işlemci kaydedicilerinde yapılır.

```
#include <stdio.h>
int main() {
        register int k, sum;
        //döngü bloğu olmayan for talimatı örneği sonunda noktalı virgül var!
        for(k = 1, sum = 0; k < 1000; sum += k, k++);
        printf("%d\n",sum); //Çıktı:499500
}
```

Öncelikle C standardı tek bir değişkenle birden fazla depolama sınıfının kullanılmasını yasaklar.
Aynı değişkene hem `static` hem `extern` niteleyicisi kullanılamaz.

Depolama sınıfları aşağıdaki tabloda özetlenmiştir; Tablodaki **çöp** (garbage) kavramı tahsis edilen bellek içeriği o anda ne ise değişkenin o değeri alması anlamındadır.

Depolama sıfatı	Bellek bölgesi	Başlangıç Değeri	Değişken Kapsamı (scope)	Değişken Ömrü (lifetime)
`auto`	Yığın bellek (stack segment)	Çöp	Bulunduğu blok	Bulunduğu blok süresince
`static`	Veri bellek (data segment)	Sıfır	Bulunduğu Blok veya Bulunduğu Dosya	Program süresi
`extern`	Veri bellek (data segment)	Sıfır	Evrensel (global)	Program süresi
`register`	İşlemci Kaydedicileri	Çöp	Bulunduğu blok	Bulunduğu blok süresince

Tablo 7.3: Depolama Sınıfları

7.6 Evrensel ve Yerel Değişken

C dilinde bir değişkene her yerden erişilmek istenilen değişkenlere **evrensel değişken** (global variable) olarak tanımlanır. Bu değişkenler herhangi bir kod bloğu içinde tanımlanmazlar. Genel olarak kaynak kodun başında tanımlanırlar. Sonra kodun her yerinde kullanılabilirler. Eğer kodumuz birden çok kaynak kod dosyasından oluşacak ise diğer tüm dosyalarda bu değişken `extern` olarak tanımlanır.

Blok içinde tanımlanmış tüm değişkenler, o blok içinden erişilebilen **yerel değişkenlerdir** (local variable). Yerel değişkenler otomatik depolama sınıfında (auto storage class) yer alır ve hepsi yığın bellekte (stack segment) yer alır.

```
1 #include <stdio.h>
```

```
2  int evrenselDegisken=10; /* evrenselDeğişken bu noktadan sonra her yerde kullanılabilir. */
3  void fonksiyon();
4  int main() {
5          int yerelDegisken=20; /* Bu yerel değişken sadece main bloğu içinde kullanılabilir. */
6          printf("evrensel:%d, yerel:%d\n",evrenselDegisken,yerelDegisken);
7          evrenselDegisken++; // evrensel değişken değiştiriliyor.
8          yerelDegisken--; // yerel değişken değiştiriliyor.
9          printf("evrensel:%d, yerel:%d\n",evrenselDegisken,yerelDegisken);
10         fonksiyon();
11         return 0;
12 }
13 void fonksiyon() {
14         int fonksiyonYerelDegiskeni=30;
15         evrenselDegisken++;
16         printf("evrensel:%d, fonsiyonYerel:%d\n",evrenselDegisken,fonksiyonYerelDegiskeni);
17         if (fonksiyonYerelDegiskeni==30) {
18                 int evrenselDegisken=100; /* Artık 2. satırdaki evrensel değişkene bu blokta
                    ↪ ulaşılamaz. */
19                 printf("evrensel:%d, fonsiyonYerel:%d\n",
                    ↪ evrenselDegisken,fonksiyonYerelDegiskeni);
20         }
21 }
22 /*Program Çıktısı:
23 evrensel:10, yerel:20
24 evrensel:11, yerel:19
25 evrensel:12, fonsiyonYerel:30
26 evrensel:100, fonsiyonYerel:30
27 */
```

Burada fonksiyon içinde `if` bloğunda 18. satırda yer alan değişken her ne kadar `evrenselDegişken` olarak kimliklendirilse de yerel değişkendir ve sadece bu `if` bloğu içinde geçerlidir. Bu yerel değişken 2. satırdaki değişkeni gölgelemiştir. Yani tanımlamada mantıksal hata (logical error) yapılmıştır.

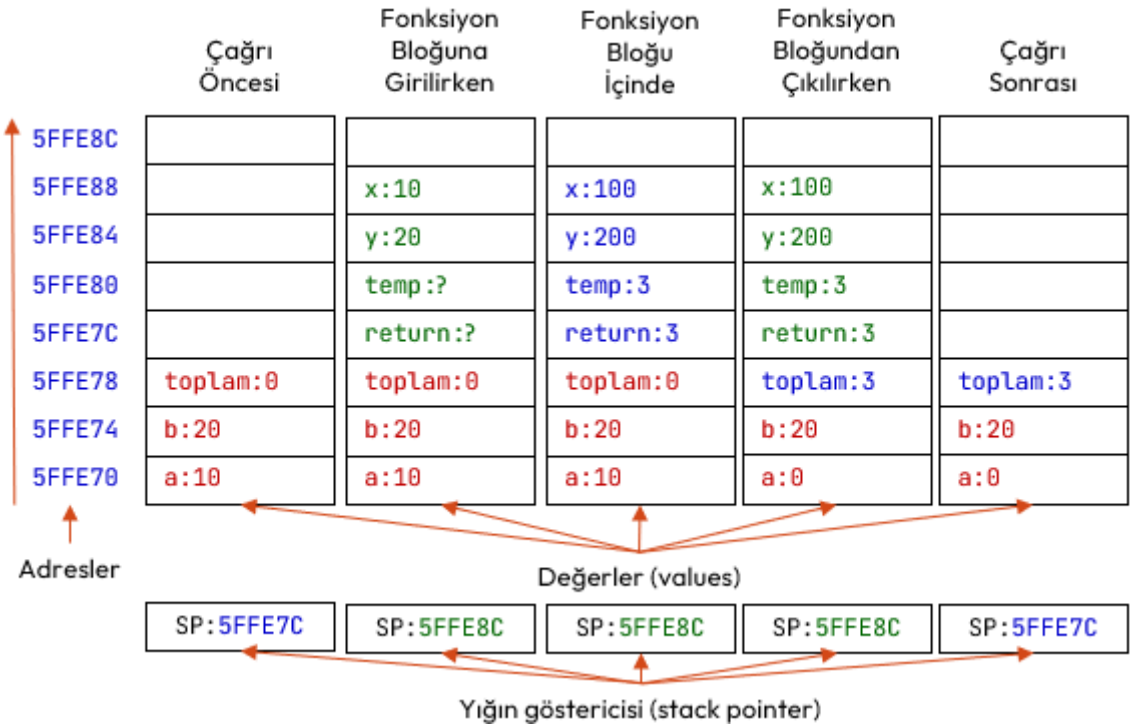
7.7 Değerle Çağırma

Yerel değişkenlerin yığın bellekte tutulduğu bir üst başlıkta anlatılmıştı. Fonksiyon blokları içindeki değişkenler de yerel değişkenlerdir (local variable). Bir fonksiyon çağrıldığında değişkenlerin bellekteki durumu ve değişimi aşağıdaki programdan anlaşılabilir;

```
#include <stdio.h>
int topla(int, int); /* İki tamsayıyı toplayan fonksiyon bildirimi/prototipi */
int main () {
        /* main() fonksiyonu içinde geçerli yerel (local) değişkenler*/
        int a = 10;
        int b = 20;
        int toplam = 0;
        printf("çağrı öncesi a: %d, b:%d, toplam:%d\n",a,b,toplam);
        toplam = topla(a, b);
        printf("çağrı sonrası a: %d, b:%d, toplam:%d\n",a,b,toplam);
}
int topla(int x, int y) { /* iki tamsayıyı toplayan fonksiyon tanımı*/
        /* Topla fonksiyonu yerel değişkenler */
        int temp= x+y;
        x=100; y=200;
        /* Burada değiştirilenler parametre değişkenleridir. main() içindeki yerel değişkenler
         ↪ değildir. */
        printf("çağrı içinde x: %d, y:%d\n",x,y);
        return temp;
}
/* Program çalıştırıldığında:
çağrı öncesi a:10, b:20 toplam:0
çağrı içinde x:100, y:200
çağrı sonrası a:10, b:20 toplam:30
```

```
*/
```

Yukarıda örneği verilen programda `topla` fonksiyonunun çağırma sürecindeki yığın bellek bölgesindeki (stack segment) değişimi Şekil 7.3'de gösterilmektedir.



Şekil 7.3: Fonksiyon Çağırma Sürecinde Yığın Bellek

Yukarıdaki kod şu şekilde açıklanabilir;

- İcra sırası `topla(a,b)` fonksiyonuna geldiğinde fonksiyon çağrılmadan önce `main()` fonksiyonu yerel değişkenleri olan `a`, `b` ve `toplam` değişkenleri yığın belleğe (stack segment) itilmiştir (push).
- `topla(a,b)` fonksiyonu çağrıldığı anda `a` ve `b` **değişkenlerinin değerleri**, `x` ve `y` **parametrelerine kopyalanarak fonksiyona geçirilir**. Değerler parametrelere kopyalandığından buna **değerle çağırma** (call by value) adı verilir.
- `topla(a,b)` fonksiyonu icra edilirken fonksiyon yerelindeki `temp` ve parametre değişkenleri `x`, ve `y` istenildiği gibi değiştirilebilir. Geri dönüş değeri de belirlenir.
- `topla(a,b)` fonksiyonu içine bir başka fonksiyon da çağrılabilirdi. Böyle bir durumda `x`, `y` ve `temp` değerleri yığın belleğe itilir ve çağrılan fonksiyon icra edilirdi. Böylece iç içe çağrılan her fonksiyonda yığın bellek büyüyeceği görülmektedir. Program derlendiğinde yığın bellek (stack segment) için sınırlı bir bellek bölgesi ayrılır. İç içe çok fonksiyon çağrıldığında bu bellek sınırı aşılabilir. Bu durumda program **yığın taşması** (stack overflow) hatası verecektir.
- `toplam=topla(a,b)` fonksiyon bloğundan çıkılırken geri dönüş değeri çağrı ortamındaki `toplam` değişkenine kopyalanır ve fonksiyon için yığın belleğe itilen değerler geri çekilir (pop).

7.8 Özyinelemeli Fonksiyonlar

Özyineleme (recursion), bir fonksiyonun kendi çağrısını yapma tekniğidir. Bu teknik, karmaşık sorunları çözülmesi daha kolay basit sorunlara ayırmanın bir yolunu sağlar. Özyineleme, yineleme (iteration), döngüler (loop) veya tekrar yerine geçebilecek çok güçlü bir tekniktir. Özyinelemeli algoritmalarda, tekrarlar fonksiyonun kendi kendisini kopyalayarak çağırması ile elde edilir. Bu kopyalar işlerini bitirdikçe kaybolur.

Bir problemi özyineleme ile çözmek için problem iki ana parçaya ayrılır;

1. Cevabı kesin olarak bildiğimiz temel durum (base case)
2. Cevabı bilinmeyen ancak cevabı yine problemin kendisi kullanılarak bulunabilecek durum.

Faktöriyel örneğini inceleyelim; Matematikte, negatif olmayan bir tam sayı olan n sayısının faktöriyeli, $n!$ ile gösterilir ve n sayısına eşit ve küçük olan tüm pozitif tam sayıların çarpımıdır.

$$n! = n \times (n-1) \times (n-2) \times \cdot \times 3 \times 2 \times 1$$

Sıfır sayısı da pozitif bir sayıdır ama sonucun sıfır çıkmaması için **`0` sayısının faktöriyeli `1` kabul edilir**. Bu

aslında faktöriyel problemi için **temel durumdur** (base case) ve **boş ürün** (emty product) kuralı olarak bilinir. Aşağıda faktöriyel için girilen sayıdan sonra tüm sayıların çarpımıyla hesaplayan bir fonksiyon yazılmıştır.

```
#include <stdio.h>
unsigned int faktoriyel(unsigned int n)
{
        int sonuc=1,indis;
        for (indis=n;indis>0;indis--)
                sonuc=sonuc*indis;
        return sonuc;
}
int main() {
        unsigned int sayi,sonuc; //Pozitif tamsayı yanımı yapılmış değişkenler.
        printf("Pozitif Bir Sayı Giriniz:");
        scanf("%d",&sayi);
        sonuc=faktoriyel(sayi);
        printf("%d!=%d\n",sayi,sonuc);
        return 0;
}
```

Aslında negatif olmayan bir tam sayı olan sayının faktöriyeli, kendisi ile kendisinden bir küçük olan sayının faktörüyeli ile çarpımı şeklinde yazılabilir. Yani problemin tanımı yine kendisini içerir:

$$n! = n \times (n-1)!$$

Bu durumda faktöriyel fonksiyonu aşağıdaki şekilde **özyinelemeli** (recursive) tanımlanabilir;

$$n! = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot (n-1)! & \text{eğer } n > 0 \end{cases}$$

Aynı faktöriyel fonksiyonu **parametreli olarak** aşağıdaki şekilde yine özyinelemeli olarak tanımlanabilir;

$$f(n) = \begin{cases} 1 & \text{eğer } n = 0 \\ n \cdot f(n-1) & \text{eğer } n > 0 \end{cases}$$

Buna örnek olarak aşağıdaki hesabı verebiliriz;

$$5! = 5 \times 4! = 5 \times 4 \times 3! = 5 \times 4 \times 3 \times 2! = 5 \times 4 \times 3 \times 2 \times 1! == 5 \times 4 \times 3 \times 2 \times 1 \times 0! = 120$$

Özyinelemeli (recursive) olarak ise faktöriyel fonksiyonu aşağıdaki şekilde kodlanabilir;

```
#include <stdio.h>
unsigned int faktoriyel(unsigned int n)
{
        if (n==0)
                return 1;
        else
                return n*faktoriyel(n-1); //Kendi kendini çağırma!
}
int main() {
        unsigned int sayi,sonuc; //Pozitif tamsayı yanımı yapılmış değişkenler.
        printf("Pozitif Bir Sayı Giriniz:");
        scanf("%d",&sayi);
        sonuc=faktoriyel(sayi);
        printf("%d!=%d\n",sayi,sonuc);
        return 0;
}
```

Bir başka örnek olarak, girilen taban ve üs ile kuvvet hesaplama fonksiyonu verilebilir. Bu fonksiyonun tanımı aşağıdaki şekilde yazılabilir;

$$f(taban, us) = \begin{cases} 0 & \text{eğer } taban = 0 \\ 1 & \text{eğer } us = 0 \\ taban \cdot f(taban, us-1) & \text{eğer } taban > 0 \text{ ve } us > 0 \end{cases}$$

Buna ilişkin kod aşağıda verilmiştir;

```
#include <stdio.h>
int kuvvet(int taban, int us) {
        if (taban == 0)
```

```
                return 0;  //Taban 0 ise 0 dön
        if (us == 0)
                return 1; // Üs 0 ise 1 dön
        return taban*kuvvet(taban, us - 1); //Kendi kendini çağırma!
}
int main(){
        int taban,us;
        printf("Tabanı Giriniz:");
        scanf("%d",&taban);
        printf("Üssü Giriniz:");
        scanf("%d",&us);
        printf("%d uzeri %d = %d", taban,us, kuvvet(taban,us) );
        return 0;
}
/*
Tabanı Giriniz:3
Üssü Giriniz:6
3 uzeri 6 = 729
*/
```

7.9 Satır İçi Fonksiyonlar

Fonksiyon Çağırma Süreci başlığında anlatıldığı üzere fonksiyonlar çağrılırken sürekli yığın belleğe (stack segment) it-çek (push-pop) yapılır. Bazı durumlarda bu işlem performans gereği istenmez. Bu durumda fonksiyonlar, **satır içi fonksiyon** (inline function) olarak tanımlanır. 2017 C sürümü ile gelen bu özellik, fonksiyona `inline` sıfatı (inline specifier) eklenerek kullanılır.

```
#include <stdio.h>
inline int topla (int op1, int op2) {
        int toplam= op1+op2;
        return toplam;
}
int main() {
        int x=10, y=15, sonuc;
        sonuc = topla(x,y);
        printf("%d+%d=%d",x,y,sonuc);
        return 0;
}
```

Bu kod aşağıdaki şekle çevrilmiş gibi derlenir;

```
#include <stdio.h>
int main() {
        int x=10, y=15, sonuc;
        {
                int toplam= op1+op2;
                sonuc = toplam;
        }
        printf("%d+%d=%d",x,y,sonuc);
        return 0;
}
```

Bir fonksiyon **aşağıda belirtilen durumlarda satır içi fonksiyon olarak derlenmez;**

1. Bir döngü içeriyorsa!
2. Statik değişkenler içeriyorsa!
3. Özyinelemeli ise!
4. Dönüş türü `void` haricinde olup `return` ifadesi işlev gövdesinde mevcut değilse!
5. `switch` veya `goto` talimatı içeriyorsa!

Bu tip fonksiyon tanımlamadaki amacımız, **fonksiyon çağırma yükünün azaltılması için işlemci kaydedicilerinin daha çok kullanılmasıdır. Bu da icra hızını yükseltir.**

7.10 Düşün ve Çöz

- **Düşün**: `static` bir yerel değişken ile `auto` bir yerel değişkenin bellek ömrü ve kapsamı arasındaki farkları karşılaştırınız.
- **Düşün**: Fonksiyonlara parametre aktarırken değerle çağırma (call by value) yönteminin yığın bellek kullanımına etkisini açıklayınız.
- **Düşün**: Özyinelemeli fonksiyonlarda temel durum (base case) unutulursa bellek seviyesinde ne gerçekleşir? Yığın taşması (stack overflow) nedir?
- **Çöz**: Bir tam sayının basamak değerleri toplamını özyinelemeli olarak hesaplayan fonksiyonu yazınız.
- **Çöz**: `extern` anahtar kelimesini kullanarak, iki farklı kaynak dosyasında (.c) ortak kullanılan bir evrensel değişken örneği kurgulayınız.